Project Proposal: Protocol Police

Extensible typestate verification for MiniScala

Filippo Nardon, Giovanni Maria Zanchetta

November 4, 2026

Contents

Idea **1**

Background and related work **1**

What the analyzer checks **2**

Adapting MiniScala **3**

Plan **3**

Evaluation **4**

Working method, AI and testing **4**

References **4**

Idea

Many APIs only work correctly if their operations are called in the right order. A file has to be opened before it can be read and should be closed before the program ends; a database transaction has to be started before it is used and should end either in a commit or in a rollback. These rules are usually written down only in the documentation, and breaking them is a classic source of bugs: leaked resources, reads on closed streams, transactions left half-finished.

For our project we want to build *Protocol Police*, a static analyzer for MiniScala that checks whether a program respects such usage protocols. Whoever writes a class describes its protocol directly in the code through annotations: the constructor states in which state a new object starts and which states count as properly finished, and each method states from which state it may be called and which state it leads to. The analyzer then looks at the program that *uses* these classes and checks every call against the declared protocol. Since the protocol lives in the annotations and not in the analyzer, adding a new protocol should only require writing a new annotated class.

The underlying concept is typestate [Strom and Yemini 1986]. We do not expect to invent a new algorithm. What we want to do is bring together techniques from the literature and from the course into one working verifier for MiniScala, adapt them where the language requires it, and measure how well the result performs.

Background and related work

Typestate has been studied for a long time, both as an extension of type systems and as a whole-program analysis, and our design borrows from both sides. Fugue [DeLine and Fähndrich 2004] and Plural [Bierhoff and Aldrich 2007] attach typestate pre- and postconditions to the methods of a class, and the Resource Leak Checker of the Checker Framework [Kellogg et al. 2021] verifies that resources get closed based on annotations in the libraries. Our annotations follow the same idea. The main difference is that these tools check programs modularly, so client code needs annotations as well, whereas we analyze the whole program and only the protocol classes have to be annotated.

On the analysis side, three papers shaped our plan. Hallem et al. [2002] show how protocol-specific state machines can be kept apart from a reusable analysis engine, which is the kind of extensibility we are after; their system is built for bug finding, though, and deliberately accepts missing some bugs, while we want a conservative analysis. ESP [Das et al. 2002] demonstrates that keeping track of which branch outcomes go together with which protocol states removes many false alarms, at a reasonable cost for protocols such as file handling. Fink et al. [2008] make clear that typestate verification is only as good as the alias information behind it: a call can update an object's state precisely only if the analysis knows which object the call refers to.

The basic machinery comes from the course [Møller and Schwartzbach] and the work it builds on: monotone frameworks, interprocedural analysis with call-string or functional contexts [Sharir and Pnueli 1981; Reps et al. 1995], inclusion-based points-to analysis [Andersen 1994] and control-flow analysis [Shivers 1991]. SPA already uses a fixed open/close protocol as an example; we want to go further and handle many objects at once, protocols declared by the user, and calls across functions. Our aim is therefore to combine typestate analysis, control-flow analysis, pointer analysis, context sensitivity and path sensitivity in a single analyzer. How far we can push each of them will depend on what the protocols and the supported language features turn out to need, and we expect to add smaller supporting analyses along the way, for instance a simple value analysis to decide the return-value conditions described in the next section.

What the analyzer checks

We want the analysis to be conservative: if the analyzer says that a program uses a protocol correctly, this should hold for every execution, at least within the part of MiniScala we support. Every use of an annotated class is checked against two requirements. Each method must be called in a state that allows it, and when the program ends every protocol object must be in one of its final states. The second check looks at the state the object is in at the end, so a file that is closed and then reopened has to be closed again.

Being conservative means that the analyzer cannot always decide whether something is wrong, so we distinguish three kinds of results. A requirement is *verified* when it holds on all executions. It produces an *error* when a violation is certain, and a *warning* when the analysis cannot rule a violation out, for example when a file may be either open or closed at a call to `read()` depending on which branch was taken before.

Some protocols depend on what a method returns. A commit may succeed or fail, and the state of the transaction afterwards depends on the result. We let annotations express this with an ordered list of cases on the return value:

```
@protocol(init = Idle, final = {Idle, Committed})
class Transaction() {
  @from(Idle) @to { case _ => Active }
  def begin(): Unit = ()

  @from(Active) @to { case 0 => Committed; case n if n > 0 => Active; case _ => Failed }
  def commit(): Int = 0

  @from(Active) @to { case _ => Idle }
  @from(Failed) @to { case _ => Idle }
  def rollback(): Unit = ()
}
```

The syntax here is only a sketch, and we will fix the details during the first week. The analyzer usually does not know what `commit()` will return, so it keeps every outcome that cannot be excluded and remembers which result belongs to which state. When the program later tests the result, the analysis can make use of that:

```
val status = t.commit();
if (status == 0) {
  // status is 0, so t is Committed for sure
} else {
  t.rollback()   // here t is Active or Failed: both allow rollback
}
```

If the analysis lost the connection between `status` and the state of `t`, it would only know that `t` is in one of three states in both branches, and it would raise warnings on a correct program. Keeping these correlations is where path

sensitivity becomes useful.

Adapting MiniScala

We plan to reuse as much as possible of the provided MiniScala implementation, in particular the parser, the type checker, the interpreter, the intermediate representation and the CFG construction. Three things are missing for our project.

The first is the annotations themselves. We will extend the lexer, the parser and the AST so that class declarations can carry the initial and final states and method declarations can carry transitions. The type checker will reject annotations that refer to unknown states or contain ill-typed conditions, while the interpreter simply ignores them.

The second is a way to split a program over several files. MiniScala programs currently consist of a single file, which would force every test program to repeat the protocol classes it uses. We will add a minimal `import "path"` directive: an imported file may only contain class declarations, each file is loaded once, cyclic imports are rejected, and the imported classes are added to the outer block of the program before type checking. Source positions will also record the file name, so that error messages point to the right place.

The third is the libraries. MiniScala has no files, transactions or networking, so we will write stub classes whose methods only pretend to do the work, for instance a `read()` that always returns the same number. For our purpose this is enough, since the analysis is about how an API is used and not about how it is implemented. Each protocol gets its own library file that programs can import:

```
import "lib/file.s"
{
  val f = new File("data.txt");
  f.open();
  f.read()          // error: f definitely ends Open, which is not a final state
}
```

We will start from a small core of the language (values, local variables, objects and references, annotated calls and conditionals) and then add loops, user-defined functions and aliasing. Higher-order functions and closures, especially closures that capture mutable variables, are something we would like to support but do not consider part of the minimum.

Plan

We expect to spend around 60 to 80 hours in total on the project. The schedule below is a guideline rather than a fixed plan: after each step we want a working and tested analyzer, and we will decide what comes next based on how far we got.

Period	Milestone
Nov 6–12	Annotation syntax, imports, file-aware positions and the `File` stub. Intraprocedural typestate check on straight-line code and conditionals, including the final-state check, with tests.
Nov 13–19	Pointer analysis for aliases, loops, function calls with call resolution, and context sensitivity.
Nov 20–26	Path-sensitive refinements (return values correlated with states), transactions as a second protocol, broader language coverage.
Nov 27–Dec 4	Evaluation and fixes; presentation on December 3 and report on December 4.

The first week is mostly about the front end, so we keep the analysis itself narrow there and leave aliasing, loops and function calls for the second week.

Evaluation

Alongside the analyzer we will build a set of benchmark programs based on protocols most programmers know: files, modelled on Java's `Closeable`, transactions, modelled on SQL's `BEGIN`, `COMMIT` and `ROLLBACK`, and, if there is time, network connections in the style of Java's `Socket`. These are small MiniScala models rather than faithful copies of the real APIs. The suite will contain short programs that each isolate one feature, such as a branch, a loop, an alias, a function call or a test on a return value, together with a few larger programs that combine them.

With these benchmarks we want to answer three questions. First, can new protocols really be added through annotations alone, without touching the analyzer? Second, how precise is the analysis: how many correct programs does it verify, how many violations does it report as errors, and how many warnings does it raise on correct code? Since we aim for a conservative analysis, we will also check that no incorrect program is ever verified. Third, how much does it cost, in running time and possibly memory, as programs grow and use more features? Where it is feasible we will also run the analysis with and without context and path sensitivity, to see what the extra precision actually buys and what it costs.

Working method, AI and testing

We will work in short iterations, following an agile process supported by the BMad method: we keep a prioritized backlog, deliver small pieces that can be tested on their own, and regularly look at the results to decide what to do next. Most of the work will be done through pair programming, switching between who writes the code and who reviews it, so that both of us follow every design decision and read every piece of generated code as it is written.

We plan to use AI throughout the project, mainly to explore the literature, to explain algorithms we are not yet familiar with, to help with the implementation and to generate variations of our test programs. We will not take generated code at face value: it is reviewed during pair programming and checked against expected results that we work out ourselves. We will also keep short notes on where the AI helped, where it got things wrong and how we noticed, which we will use for the reflection in the final report.

Testing will grow together with the analyzer. We will check that malformed or inconsistent annotations are rejected, and for each feature we will write small programs whose expected result (verified, warning or error) we determine by hand. As more of the language is supported we will combine features in the same program, and we will keep all earlier tests as regression tests. A passing test suite does not prove that the analysis is sound, but it gives us confidence that the implementation does what we intend.

References

- L. O. Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994.
- K. Bierhoff and J. Aldrich. Modular typestate checking of aliased objects. *OOPSLA*, 2007.
- M. Das, S. Lerner and M. Seigle. ESP: Path-sensitive program verification in polynomial time. *PLDI*, 2002.
- R. DeLine and M. Fähndrich. Typestates for objects. *ECOOP*, 2004.
- S. J. Fink, E. Yahav, N. Dor, G. Ramalingam and E. Geay. Effective typestate verification in the presence of aliasing. *ACM TOSEM*, 2008.
- S. Hallem, B. Chelf, Y. Xie and D. Engler. A system and language for building system-specific, static analyses. *PLDI*, 2002.
- M. Kellogg, N. Shadab, M. Sridharan and M. D. Ernst. Lightweight and modular resource leak verification. *ESEC/FSE*, 2021.
- A. Møller and M. I. Schwartzbach. *Static Program Analysis*. Aarhus University.
- T. Reps, S. Horwitz and M. Sagiv. Precise interprocedural dataflow analysis via graph reachability. *POPL*, 1995.
- M. Sharir and A. Pnueli. Two approaches to interprocedural data flow analysis. In *Program Flow Analysis: Theory and Applications*, 1981.
- O. Shivers. *Control-Flow Analysis of Higher-Order Languages*. PhD thesis, Carnegie Mellon University, 1991.
- R. E. Strom and S. Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE TSE*, 1986.